Module 2 — SQL joins, aggregation and intermediate querying

Learning outcomes

Choose the correct join, predict row multiplication, aggregate at the correct grain, use CASE, subqueries, CTEs and set operations.

1. Join types

• INNER JOIN: only matching rows.

• LEFT JOIN: all left rows; unmatched right columns become NULL.

• RIGHT JOIN: all right rows and matches.

• FULL OUTER JOIN: matched and unmatched rows from both sides.

• CROSS JOIN: every left row paired with every right row.

• Self join: a table joined to itself.

2. The join multiplication problem

Suppose order 17 has four item rows. `orders JOIN order_items` returns four rows for order 17. Therefore `SUM(orders.order_amount)` after this join counts the order amount four times.

Correct approaches:

• aggregate at order grain before joining;

• calculate item-level revenue from item rows;

• use `EXISTS` when only checking whether a related row exists.

3. Aggregation

`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.

`GROUP BY` forms groups. `HAVING` filters groups. WHERE filters source rows before grouping.

4. CASE and conditional aggregation

```
SELECT customer_id,
       SUM(CASE WHEN status = 'PAID' THEN amount ELSE 0 END) AS paid_amount,
       SUM(CASE WHEN status = 'REFUNDED' THEN amount ELSE 0 END) AS refunded_amount
FROM payments
GROUP BY customer_id;
```

5. Subqueries and EXISTS

Use a scalar subquery for one value, a correlated subquery for row-dependent logic, and `EXISTS`/`NOT EXISTS` for existence checks. `NOT IN` can behave unexpectedly if the subquery returns NULL.

6. CTEs

A Common Table Expression names a query result for readability and decomposition. It is not guaranteed to be materialised; optimiser behaviour differs by database/version.

7. UNION vs UNION ALL

`UNION` removes duplicates; `UNION ALL` preserves all rows and is usually cheaper. Both require compatible column counts/types.

Worked exercise schema

```
departments(department_id, department_name)
employees(employee_id, employee_name, department_id, salary, manager_id)
```

Guided tasks and solutions

1. List employees with department names.

```
SELECT e.employee_id, e.employee_name, d.department_name
FROM employees e
```

```
LEFT JOIN departments d
  ON d.department_id = e.department_id;
```

2. Count employees per department, including departments with zero employees.

```
SELECT d.department_id, d.department_name,
       COUNT(e.employee_id) AS employee_count
FROM departments d
LEFT JOIN employees e
  ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name;
```

Use `COUNT(e.employee_id)`, not `COUNT(*)`, because the unmatched LEFT JOIN row still exists.

3. Find departments with average salary above 80000.

```
SELECT d.department_name, AVG(e.salary) AS avg_salary
FROM departments d
JOIN employees e ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name
HAVING AVG(e.salary) > 80000;
```

4. Find employees earning more than their department average.

```
SELECT e.*
FROM employees e
WHERE e.salary > (
  SELECT AVG(e2.salary)
  FROM employees e2
  WHERE e2.department_id = e.department_id
);
```

5. Find employees who manage at least one other employee.

```
SELECT m.employee_id, m.employee_name
FROM employees m
WHERE EXISTS (
  SELECT 1 FROM employees r
  WHERE r.manager_id = m.employee_id
);
```

6. Return employees and their manager names.

```
SELECT e.employee_name AS employee,
       m.employee_name AS manager
FROM employees e
LEFT JOIN employees m ON m.employee_id = e.manager_id;
```

Independent exercises

7. Find the second-highest distinct salary.

8. Find employees whose salary is above the company average.

9. Find employees who do not belong to a valid department.

10. Return department headcount and payroll, including empty departments.

11. Given `web_events(user_id, event_type)`, count unique users who performed `signup` and `purchase` separately.

12. Explain when `UNION ALL` is correct for appending daily source extracts.

Solutions

```
-- 7: handles ties by returning the second distinct value
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- 8
SELECT *
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- 9: orphan references (if FK is not enforced)
SELECT e.*
FROM employees e
LEFT JOIN departments d ON d.department_id = e.department_id
WHERE e.department_id IS NOT NULL
  AND d.department_id IS NULL;

-- 10
SELECT d.department_name,
       COUNT(e.employee_id) AS headcount,
       COALESCE(SUM(e.salary), 0) AS payroll
FROM departments d
LEFT JOIN employees e ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name;

-- 11
SELECT
  COUNT(DISTINCT user_id) FILTER (WHERE event_type = 'signup') AS signups,
  COUNT(DISTINCT user_id) FILTER (WHERE event_type = 'purchase') AS purchasers
FROM web_events;
```

`FILTER` is PostgreSQL syntax. A portable alternative is `COUNT(DISTINCT CASE WHEN ... THEN user_id END)`.

Common mistakes to avoid

• Joining two one-to-many tables before aggregating.

• Counting rows after a LEFT JOIN rather than counting a non-null right-side key.

• Filtering right-table columns in WHERE when you intended to preserve unmatched left rows.

• Using DISTINCT to hide a wrong join.

• Forgetting that NULL values are ignored by most aggregates.

Mini assessment (20 minutes)

Write queries for: (a) top 3 departments by payroll, (b) employees with no manager, (c) managers with at least 4 direct reports, (d) department salary range, (e) employees whose salary is within 10% of department average. Mark 2 points each for correctness and 2 for explaining assumptions.

External practice for this module

1. HackerRank SQL — Aggregation — solve at least 8 aggregation questions; include GROUP BY, HAVING, COUNT and conditional calculations.

2. LeetCode SQL 50 — complete Basic Joins, Basic Aggregate Functions and Sorting and Grouping sections.

3. DataLemur SQL interview questions — choose 8 questions tagged around joins, aggregations, GROUP BY or business metrics. Attempt before opening hints.

4. PostgreSQL Exercises — complete the joins and aggregates sections against its sample database.

Exit requirement: solve 12 unseen join/aggregation questions. For each, state the grain before writing SQL and explain why a LEFT JOIN does or does not preserve unmatched rows. Include at least two questions where a one-to-many join could inflate a metric.